



National Laboratory of the Rockies

2 — SDOM Inputs and Data Loading

This module explains how SDOM input folders are structured and how to inspect the Python dictionary produced by `load_data` using the sample scenario.

Learning objectives

- Identify the core CSV files used by SDOM in `data/sample_data/`.
- Load the input scenario with `sdom.load_data`.
- Explore the resulting dictionary keys, table shapes, and key parameters.
- Verify active formulations before running simulations.

Prerequisites

- You completed module 1: [training/1_sdom_install/README_m1.md](#).
- Your virtual environment is active and SDOM is installed.

Inputs and scenario used

- Scenario folder: `data/sample_data/`
- Key files used in this module:
 - `CapSolar_2050.csv`, `CapWind_2050.csv`
 - `CFSolar_2050.csv`, `CFWind_2050.csv`
 - `Data_BalancingUnits_2030(in).csv`
 - `formulations.csv`
 - `lahy_hourly_2019.csv`
 - `Load_hourly_2050.csv`
 - `Nucl_hourly_2019.csv`

- `otre_hourly_2019.csv`
- `scalars.csv`
- `StorageData_2050.csv`

Reference:

- SDOM input data guide: https://natlabrockies.github.io/SDOM/user_guide/inputs.html

Input file naming and column conventions

SDOM scenarios are folders of CSV files read by `load_data`. The expected file types and shapes are documented in the SDOM input data guide: https://natlabrockies.github.io/SDOM/user_guide/inputs.html.

In `data/sample_data/`, several files include a year suffix such as `_2050` or `_2030(in)`. These suffixes are scenario labels. They can be changed when you create a new scenario, but the file role must remain recognizable to SDOM and to the training scripts. For example, if a script expects `CFSolar_2050.csv`, then renaming it to `CFSolar_Togo2050.csv` also requires updating the script or the data-loading convention that maps file names to SDOM input tables.

The stable part of each name communicates the input role:

- `CapSolar_*` and `CapWind_*`: candidate renewable sites and investment data.
- `CFSolar_*` and `CFWind_*`: hourly capacity factors for those candidate sites.
- `StorageData_*`: storage technology costs and technical parameters.
- `Load_hourly_*`: hourly demand profile.
- `formulations.csv`: active model formulation choices.
- `scalars.csv`: global scalar parameters such as `GenMix_Target`.

Column names matter because SDOM uses them to connect tables. In hourly capacity factor files, the first column is the time index (`Hour`), and each remaining column is a site ID. Those site IDs must match the `sc_gid` values in the corresponding capacity file. For example:

- Columns in `data/sample_data/CFSolar_2050.csv` include solar site IDs such as `132876` and `132875`.
- Those same IDs must appear in the `sc_gid` column of `data/sample_data/CapSolar_2050.csv`.
- The same rule applies to `CFWind_2050.csv` and `CapWind_2050.csv`.

Storage data uses the opposite orientation: technologies are columns and parameters are rows. For example, in `data/sample_data/StorageData_2050.csv`, columns such as `Li-Ion`, `CAES`, `PHS`, and `H2` are storage technologies, while rows such as `P_Capex`, `E_Capex`, `Eff`, `Min_Duration`, and `Max_Duration` define their costs and technical limits.

Examples of safe edits:

- Updating numeric values in `P_Capex`, `E_Capex`, `Eff`, `Min_Duration`, or `Max_Duration` for an existing storage technology.
- Adding a new solar candidate if the same ID is added consistently to both `CapSolar_2050.csv` (`sc_gid`) and `CFSolar_2050.csv` (capacity-factor column).
- Changing a scenario suffix such as `_2050` only after updating any scripts or documentation that reference the original file name.

Examples of risky edits:

- Renaming `sc_gid`, `capacity`, `latitude`, or `longitude` columns in capacity files without updating SDOM's input parsing expectations.
- Adding a capacity-factor column whose ID does not exist in the matching capacity file.
- Removing rows such as `P_Capex` or `E_Capex` from `StorageData_2050.csv` when storage investment is active.

Main input files in `data/sample_data/`

The table below lists every input file currently present in `data/sample_data/`.

File	Main purpose	Key naming / column notes
<code>formulations.csv</code>	Selects which model formulations are active, such as thermal, hydro, imports, or exports.	<code>Component</code> and <code>Formulation</code> values determine which input files SDOM expects.
<code>scalars.csv</code>	Stores scalar model parameters, for example <code>GenMix_Target</code> , <code>r</code> , and <code>EUE_max</code> .	Keep parameter names stable; scripts and sweeps may reference them directly.
<code>StorageData_2050.csv</code>	Contains CAPEX costs and technical parameters for storage technologies.	Technology names are columns; rows include <code>P_Capex</code> , <code>E_Capex</code> , <code>Eff</code> , <code>Min_Duration</code> , and <code>Max_Duration</code> .
<code>Load_hourly_2050.csv</code>	Contains the hourly demand profile.	Uses a time index column and a <code>Load</code> column. Full planning runs should use 8760 hours.
<code>CapSolar_2050.csv</code>	Contains solar candidate-site data such as CAPEX components, maximum capacity, and latitude/longitude.	<code>sc_gid</code> identifies each site; these IDs must match columns in <code>CFSolar_2050.csv</code> .
<code>CFSolar_2050.csv</code>	Contains hourly capacity factors for candidate solar sites.	Column names after <code>Hour</code> must match <code>sc_gid</code> values in <code>CapSolar_2050.csv</code> .
<code>CapWind_2050.csv</code>	Contains wind candidate-site data such as CAPEX components, maximum capacity, and latitude/longitude.	<code>sc_gid</code> identifies each site; these IDs must match columns in <code>CFWind_2050.csv</code> .
<code>CFWind_2050.csv</code>	Contains hourly capacity factors for candidate wind sites.	Column names after <code>Hour</code> must match <code>sc_gid</code> values in <code>CapWind_2050.csv</code> .

File	Main purpose	Key naming / column notes
<code>Data_BalancingUnits_2030(in).csv</code>	Contains thermal balancing-unit parameters such as capacity bounds, lifetime, CAPEX, heat rate, and fuel cost.	<code>Plant_id</code> identifies each thermal unit; cost and technical columns should keep their expected names.
<code>Nuc1_hourly_2019.csv</code>	Contains nuclear generation or availability time-series data when nuclear is active.	Hourly columns should keep the expected time-series structure from the SDOM input guide.
<code>lahy_hourly_2019.csv</code>	Contains large-hydro hourly profile data.	Used with hydro formulations; keep time indexing consistent with other hourly files.
<code>otre_hourly_2019.csv</code>	Contains other-renewables hourly profile data.	Used when other-renewables data is represented as an exogenous profile.

For more detail on required files, hourly structure, and validation, see the SDOM input documentation: https://natlabrockies.github.io/SDOM/user_guide/inputs.html#required-files.

Project/file structure for this module

```
training/2_sdom_inputs/
├── README_m2.md
└── explore_data.py
```

Module suffix and script naming

- Module suffix: `m2`
- Script naming note: this data-exploration module keeps `explore_data.py` (the `run_mx.py` naming is used in simulation modules).

Step-by-step walkthrough

1. Activate the environment and run:

```
python training/2_sdom_inputs/explore_data.py
```

2. Confirm the script prints:

- The top-level dictionary keys.
- Shapes for key data tables.
- Active formulation selections.

- Sample values (e.g., `GenMix_Target`).

Full runnable script

See [training/2_sdom_inputs/explore_data.py](#).

Inspect `load_data` help from Python

This module introduces `sdom.load_data`, the function that reads an SDOM input folder into the Python data dictionary used by later modules. Use Python's built-in `help()` function to inspect its signature and docstring.

Run this from the repository root with the virtual environment active:

```
python -c "from sdom import load_data; help(load_data)"
```

Or use an interactive Python session:

```
from sdom import load_data
help(load_data)
```

Expected output from this environment:

```
Help on function load_data in module sdom.io_manager:
```

```
load_data(input_data_dir: str = './Data\')
```

```
    Load all required SDOM input datasets from CSV files in the specified
    directory.
```

```
    Reads and validates all input CSV files needed for SDOM optimization including
    VRE data, fixed generation profiles, storage characteristics, thermal units,
    scalars, and formulation specifications. Performs data consistency checks and
    filters datasets based on completeness.
```

```
    Args:
```

```
        input_data_dir (str, optional): Path to directory containing input CSV
    files.
```

```
        Defaults to './Data/'. Should contain all required files defined in
    constants.INPUT_CSV_NAMES.
```

```
    Returns:
```

```
        dict: Dictionary containing loaded and processed data with keys:
```

- 'formulations' (pd.DataFrame): Component formulation specifications
- 'solar_plants', 'wind_plants' (list): Plant IDs for VRE technologies
- 'cf_solar', 'cf_wind' (pd.DataFrame): Hourly capacity factors
- 'cap_solar', 'cap_wind' (pd.DataFrame): Plant CAPEX and capacity data
- 'load_data' (pd.DataFrame): Hourly electricity demand
- 'storage_data' (pd.DataFrame): Storage technology characteristics
- 'thermal_data' (pd.DataFrame): Thermal balancing unit parameters

```
- 'scalars' (pd.DataFrame): System-level scalar parameters
```

Raises:

`FileNotFoundError`: If any required input file is missing from `input_data_dir`.

`ValueError`: If formulation specifications are invalid.

Expected outputs

- A compact inventory of the loaded SDOM data dictionary.
- Printed table shapes for critical input tables.
- A summary of which formulations are active in the sample case.

How to validate results

- The script should complete without exceptions.
- At least the keys `scalars`, `storage_data`, and `load_data` should appear.
- `GenMix_Target` should be listed from `scalars`.

Troubleshooting

- `FileNotFoundError`: run the script from repo root, or adjust paths.
- Missing keys in printed output: verify that `data/sample_data/` has all source CSVs.
- Import errors: activate `.venv` before running Python.

References

- SDOM docs home: <https://natlabrockies.github.io/SDOM/index.html>
- Input data guide: https://natlabrockies.github.io/SDOM/user_guide/inputs.html
- API reference (core): <https://natlabrockies.github.io/SDOM/api/core.html>